

SKILL WEEK8

Session – 15

TASK 1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

A screenshot of a terminal window with a dark background. The window title bar shows 'rishill10@ASUS16X: ~/Week8_'. The terminal content shows the GNU nano 8.7.1 editor editing 'session15_task1.c'. The code is a C program for parsing combined redirection. It includes headers like <stdio.h>, <stdlib.h>, <string.h>, <unistd.h>, <fcntl.h>, <sys/types.h>, and <sys/wait.h>. It defines MAX_INPUT as 200. The main function starts by printing 'Session 15 - Combined Redirection\n' and 'Enter command with combined redirection:\n'. It then prompts 'myshell> ' and reads input into 'input' using fgets. It checks if input is NULL and returns 1 if so. It then finds the end of the input string and sets it to '\0'. It searches for a redirection operator '>' in the input string. If found, it checks for a file descriptor '&1' for combined redirection. If both are found, it prints an error message. Finally, it finds the start of the file name and skips spaces and tabs.

```
GNU nano 8.7.1 session15_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main()
{
    char input[MAX_INPUT];
    char filename[100];

    printf("Session 15 - Combined Redirection\n");
    printf("Enter command with combined redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">");
    if (redirect == NULL)
    {
        printf("Error: Output redirection not found.\n");
        return 1;
    }

    char *merge = strstr(input, "2>&1");
    if (merge == NULL)
    {
        printf("Error: Use 2>&1 for combined redirection.\n");
        return 1;
    }

    char *file_start = redirect + 1;
    while (*file_start == ' ' || *file_start == '\t')
        file_start++;
}
```

```

char *file_end = merge;
while (file_end > file_start && (*(file_end - 1) == ' ' || *(file_end - 1) == '\n'))
{
    file_end--;
}

int length = file_end - file_start;
if (length <= 0 || length >= 100)
{
    printf("Error: Invalid output filename.\n");
    return 1;
}

strncpy(filename, file_start, length);
filename[length] = '\0';

printf("\nRedirection detected:\n");
printf("stdout -> %s\n", filename);
printf("stderr -> stdout\n");

int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1)
{
    perror("Error opening output file");
    return 1;
}

printf("Output file opened successfully.\n");

pid_t pid = fork();
if (pid < 0)
{
    perror("fork failed");
    close(fd);
    return 1;
}

if (pid == 0)
{
    if (dup2(fd, STDOUT_FILENO) == -1)
    {

```

```

        perror("dup2 stdout failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    printf("This is standard output.\n");
    fprintf(stderr, "This is standard error.\n");
    printf("Both streams are merged successfully.\n");
    exit(0);
}

close(fd);

int status;
waitpid(pid, &status, 0);

printf("\nChild process completed.\n");
if (WIFEXITED(status))
{
    printf("Child exit status: %d\n", WEXITSTATUS(status));
}

FILE *fp = fopen(filename, "r");
if (fp == NULL)
{
    perror("Error reading output file");
    return 1;
}

char buffer[300];
printf("\nCombined output file contents:\n");
printf("-----\n");
while (fgets(buffer, sizeof(buffer), fp) != NULL)
{
    printf("%s", buffer);

```

```

}
printf("-----\n");
fclose(fp);

printf("Output ordering verified.\n");
printf("Combined redirection completed successfully.\n");
return 0;
}

```

```
rishill10@ASUS16X: ~/Week8_ x + v
rishill10@ASUS16X:~$ mkdir Week8_Skill
cd Week8_Skill
rishill10@ASUS16X:~/Week8_Skill$ nano session15_task1.c
rishill10@ASUS16X:~/Week8_Skill$ gcc session15_task1.c -o session15_task1
./session15_task1
Session 15 - Combined Redirection
Enter command with combined redirection:
myshe11> echo test > output.txt 2>&1

Redirection detected:
stdout -> output.txt
stderr -> stdout
Output file opened successfully.

Child process completed.
Child exit status: 0

Combined output file contents:
-----
This is standard output.
This is standard error.
Both streams are merged successfully.
-----
Output ordering verified.
Combined redirection completed successfully.
rishill10@ASUS16X:~/Week8_Skill$ |
```

TASK 2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
GNU nano 8.7.1 session15_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
    char input_file[100];
    char output_file[100];
} Command;

int parse_complex(char *input, Command commands[])
{
    int count = 0;
    char *saveptr1, *saveptr2;
    char *part = strtok_r(input, "|", &saveptr1);

    while (part != NULL && count < MAX_COMMANDS)
    {
        commands[count].argc = 0;
        commands[count].input_file[0] = '\0';
        commands[count].output_file[0] = '\0';

        char *arg = strtok_r(part, " \t\n", &saveptr2);
        while (arg != NULL && commands[count].argc < MAX_ARGS - 1)
        {
            if (strcmp(arg, "<") == 0)
            {
                arg = strtok_r(NULL, " \t\n", &saveptr2);
                if (arg != NULL) strcpy(commands[count].input_file, arg);
            }
            else if (strcmp(arg, ">") == 0)
            {
                arg = strtok_r(NULL, " \t\n", &saveptr2);
```

```

        }
        else if (strcmp(arg, ">") == 0)
        {
            arg = strtok_r(NULL, " \t\n", &saveptr2);
            if (arg != NULL) strcpy(commands[count].output_file, arg);
        }
        else
        {
            commands[count].args[commands[count].argc++] = arg;
        }
        arg = strtok_r(NULL, " \t\n", &saveptr2);
    }
    commands[count].args[commands[count].argc] = NULL;

    if (commands[count].argc == 0)
    {
        printf("Error: Empty command in pipeline.\n");
        return -1;
    }
    count++;
    part = strtok_r(NULL, "|", &saveptr1);
}
return count;
}

int main()
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 15 - Complex Pipeline Execution\n");
    printf("Enter a complex command:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");

```

```

        return 0;
    }

    int command_count = parse_complex(input, commands);
    if (command_count <= 0)
    {
        printf("Syntax validation failed.\n");
        return 1;
    }

    printf("\n---\n");
    printf("Syntax validation: SUCCESS\n");
    printf("Execution order: Left to Right\n");

    int pipes[MAX_COMMANDS - 1][2];
    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe failed");
            return 1;
        }
    }

    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < command_count; i++)
    {
        pids[i] = fork();
        if (pids[i] < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pids[i] == 0)
        {
            if (i > 0)
            {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }
            if (i < command_count - 1)

```

```

        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        if (strlen(commands[i].input_file) > 0)
        {
            int fd = open(commands[i].input_file, O_RDONLY);
            if (fd == -1)
            {
                perror("Input file error");
                exit(EXIT_FAILURE);
            }
            dup2(fd, STDIN_FILENO);
            close(fd);
        }

        if (strlen(commands[i].output_file) > 0)
        {
            int fd = open(commands[i].output_file, O_WRONLY | O_CREAT | O_APPEND);
            if (fd == -1)
            {
                perror("Output file error");
                exit(EXIT_FAILURE);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(commands[i].args[0], commands[i].args);
        perror("Execution failed");
        exit(EXIT_FAILURE);
    }
}

for (int i = 0; i < command_count - 1; i++)
{

```

```

    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < command_count; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("\nAll commands executed successfully.\n");
    printf("Complex pipeline execution completed.\n");
    return 0;
}

```

```

rishill10@ASUS16X:~/Week8_Skill$ echo -e "Line 1\nLine 2\nLine 3\nLine 4" >
input.txt
rishill10@ASUS16X:~/Week8_Skill$ ./session15_task2
Session 15 - Complex Pipeline Execution
Enter a complex command:
myshell> cat < input.txt | grep l | wc -l > output.txt

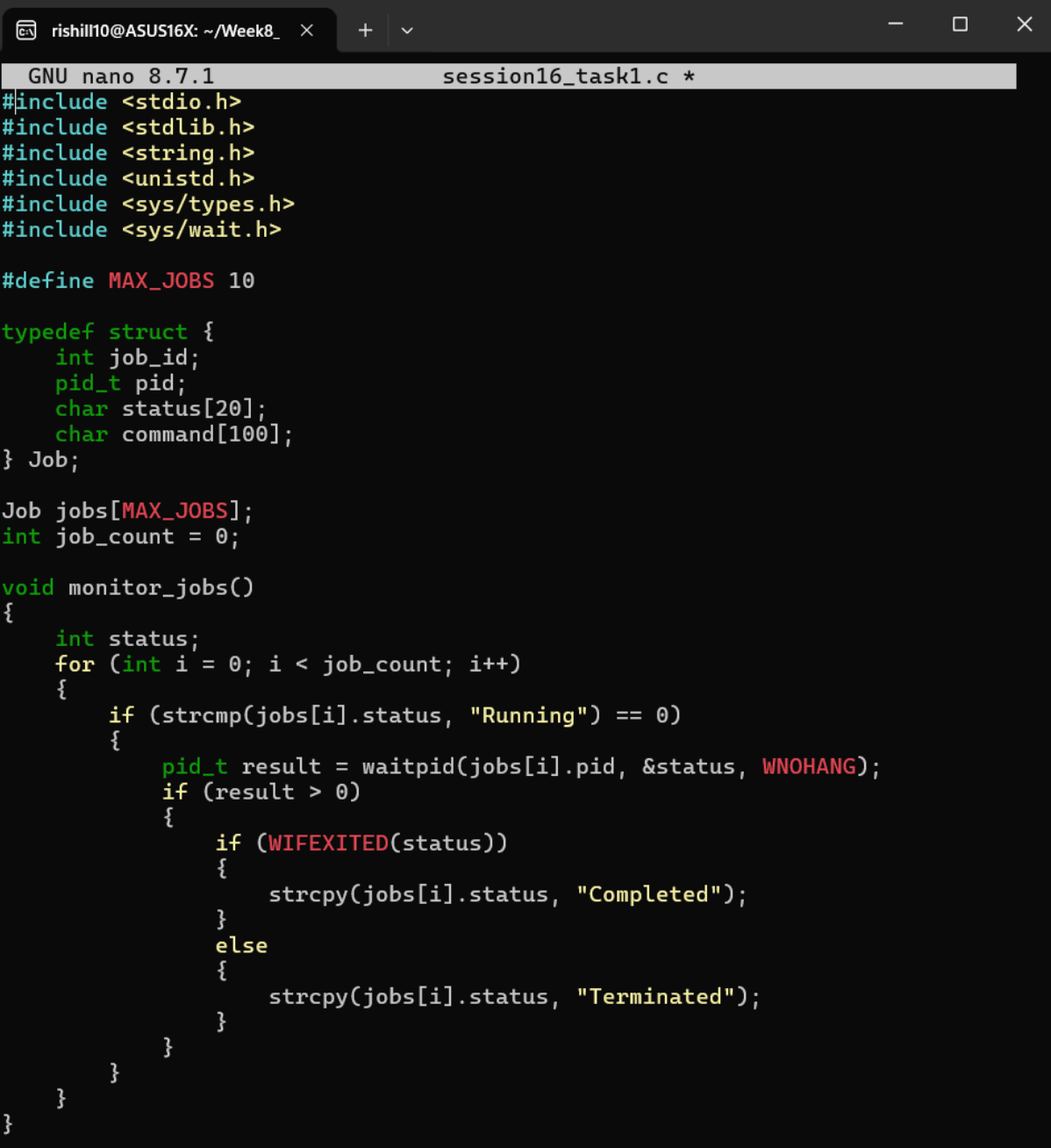
---
Syntax validation: SUCCESS
Execution order: Left to Right

All commands executed successfully.
Complex pipeline execution completed.
rishill10@ASUS16X:~/Week8_Skill$ |

```

Session – 16

TASK 1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.



```
GNU nano 8.7.1 session16_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char status[20];
    char command[100];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void monitor_jobs()
{
    int status;
    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].status, "Running") == 0)
        {
            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);
            if (result > 0)
            {
                if (WIFEXITED(status))
                {
                    strcpy(jobs[i].status, "Completed");
                }
                else
                {
                    strcpy(jobs[i].status, "Terminated");
                }
            }
        }
    }
}
```

```

void show_jobs()
{
    monitor_jobs();
    printf("\nBackground Jobs:\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\t\tCommand\n");
    printf("-----\n");
    if (job_count == 0)
    {
        printf("No background jobs.\n");
    }
    else
    {
        for (int i = 0; i < job_count; i++)
        {
            printf("[%d]\t%d\t%-10s\t%s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].status,
                    jobs[i].command);
        }
    }
    printf("-----\n");
}

void add_job(pid_t pid, const char *cmd)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        strcpy(jobs[job_count].status, "Running");
        strncpy(jobs[job_count].command, cmd, sizeof(jobs[job_count].command));
        job_count++;
    }
}

int main()
{
    char input[100];

```

```

    printf("Session 16 - Background Jobs\n");
    printf("Commands: run, jobs, exit\n");

    while (1)
    {
        monitor_jobs();

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "run") == 0)
        {
            pid_t pid = fork();
            if (pid < 0)
            {
                perror("fork failed");
            }
            else if (pid == 0)
            {
                printf("\nBackground process started. PID: %d\n", getpid());
                sleep(10);
                printf("\nBackground process completed. PID: %d\n", getpid());
                exit(0);
            }
            else
            {
                add_job(pid, "sleep 10");
                printf("Background job started: [%d] PID %d\n", job_count, pid);
                printf("Prompt returned immediately.\n");
            }
        }
        else if (strcmp(input, "jobs") == 0)
        {
            show_jobs();
        }
        else if (strcmp(input, "exit") == 0)
        {

```

```

        printf("\nWaiting for background jobs...\n");
        for (int i = 0; i < job_count; i++)
        {
            if (strcmp(jobs[i].status, "Running") == 0)
            {
                waitpid(jobs[i].pid, NULL, 0);
                strcpy(jobs[i].status, "Completed");
            }
        }
        printf("All background jobs completed.\n");
        printf("Exiting shell...\n");
        break;
    }
    else if (strlen(input) == 0)
    {
        continue;
    }
    else
    {
        printf("Invalid command: %s\n", input);
        printf("Use: run, jobs, or exit\n");
    }
}
return 0;
}

```

```

rishill10@ASUS16X: ~/Week8_ x + v
rishill10@ASUS16X:~/Week8_Skill$ nano session16_task1.c
rishill10@ASUS16X:~/Week8_Skill$ gcc session16_task1.c -o session16_task1
./session16_task1
Session 16 - Background Jobs
Commands: run, jobs, exit
myshell> run
Background job started: [1] PID 805
Prompt returned immediately.
myshell>
Background process started. PID: 805

Background process completed. PID: 805
jobs

Background Jobs:
-----
Job ID  PID    Status    Command
-----
[1]     805     Completed  sleep 10
-----
myshell> exit

Waiting for background jobs...
All background jobs completed.
Exiting shell...
rishill10@ASUS16X:~/Week8_Skill$ |

```

TASK 2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
GNU nano 8.7.1 session16_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char state[20];
    char command[100];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

void monitor_jobs()
{
    int status;
    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(job_table[i].state, "Running") == 0)
        {
            pid_t result = waitpid(job_table[i].pid, &status, WNOHANG);
            if (result > 0)
            {
                if (WIFEXITED(status))
                {
                    strcpy(job_table[i].state, "Completed");
                }
                else
                {
                    strcpy(job_table[i].state, "Terminated");
                }
            }
        }
    }
}
```

```

void show_jobs()
{
    monitor_jobs();
    printf("\nJob Table:\n");
    printf("-----\n");
    printf("Job ID\tPID\tPGID\tState\t\tCommand\n");
    printf("-----\n");
    if (job_count == 0)
    {
        printf("No jobs in table.\n");
    }
    else
    {
        for (int i = 0; i < job_count; i++)
        {
            printf("[%d]\t%d\t%d\t%-10s\t%s\n",
                    job_table[i].job_id,
                    job_table[i].pid,
                    job_table[i].pgid,
                    job_table[i].state,
                    job_table[i].command);
        }
    }
    printf("-----\n");
}

void add_job(pid_t pid, pid_t pgid, const char *cmd)
{
    if (job_count < MAX_JOBS)
    {
        job_table[job_count].job_id = job_count + 1;
        job_table[job_count].pid = pid;
        job_table[job_count].pgid = pgid;
        strcpy(job_table[job_count].state, "Running");
        strncpy(job_table[job_count].command, cmd, sizeof(job_table[job_cou
        job_count++;
    }
}

void update_jobs()
{
|

```

```

        monitor_jobs();
    }

void remove_completed_jobs()
{
    int new_count = 0;
    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(job_table[i].state, "Running") == 0)
        {
            job_table[new_count] = job_table[i];
            job_table[new_count].job_id = new_count + 1;
            new_count++;
        }
        else
        {
            printf("Removing Job [%d] from job table.\n", job_table[i].job_
        }
    }
    job_count = new_count;
}

void validate_jobs()
{
    int valid = 1;
    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].job_id <= 0 || job_table[i].pid <= 0)
        {
            valid = 0;
            break;
        }
    }
    if (valid)
        printf("All job records are valid.\n");
    else
        printf("Invalid job records found.\n");
}

int main()
{
    char input[100];

```

```

printf("Session 16 - Job Table Management\n");
printf("Commands: run, jobs, update, remove, validate, exit\n");

while (1)
{
    monitor_jobs();
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "run") == 0)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
        }
        else if (pid == 0)
        {
            setpgid(0, 0);
            printf("\nChild process started.\n");
            printf("PID: %d\n", getpid());
            printf("PGID: %d\n", getpgrp());
            sleep(8);
            printf("\nChild process completed.\n");
            exit(0);
        }
        else
        {
            setpgid(pid, pid);
            add_job(pid, pid, "sleep 8");
            printf("Job created: [%d] PID: %d PGID: %d\n", job_count, p
        }
    }
    else if (strcmp(input, "jobs") == 0)
    {
        show_jobs();
    }
}

```

```

        show_jobs();
    }
    else if (strcmp(input, "update") == 0)
    {
        update_jobs();
        printf("Job states updated successfully.\n");
        show_jobs();
    }
    else if (strcmp(input, "remove") == 0)
    {
        update_jobs();
        remove_completed_jobs();
        printf("Job table cleanup completed.\n");
    }
    else if (strcmp(input, "validate") == 0)
    {
        validate_jobs();
    }
    else if (strcmp(input, "exit") == 0)
    {
        printf("\nCleaning up jobs...\n");
        for (int i = 0; i < job_count; i++)
        {
            if (strcmp(job_table[i].state, "Running") == 0)
            {
                kill(-job_table[i].pgid, SIGTERM);
                waitpid(job_table[i].pid, NULL, 0);
            }
        }
        printf("All resources cleaned up.\n");
        printf("Exiting...\n");
        break;
    }
    else if (strlen(input) == 0)
    {
        continue;
    }
    else
    {
        printf("Invalid command.\n");
        printf("Use: run, jobs, update, remove, validate, exit\n");
    }
}

```

```

    {
        printf("Invalid command.\n");
        printf("Use: run, jobs, update, remove, validate, exit\n");
    }
}
return 0;
}
|

```

```

rishill10@ASUS16X: ~/Week8_ × + ▾
rishill10@ASUS16X:~/Week8_Skill$ nano session16_task2.c
rishill10@ASUS16X:~/Week8_Skill$ gcc session16_task2.c -o session16_task2
./session16_task2
Session 16 - Job Table Management
Commands: run, jobs, update, remove, validate, exit
myshell> run
Job created: [1] PID: 877 PGID: 877
myshell>
Child process started.
PID: 877
PGID: 877
jobs

Job Table:
-----
Job ID  PID    PGID   State   Command
-----
[1]     877     877    Running sleep 8
-----

myshell> update
Job states updated successfully.

Job Table:
-----
Job ID  PID    PGID   State   Command
-----
[1]     877     877    Running sleep 8
-----

myshell> remo
Child process completed.
remove
Invalid command.
Use: run, jobs, update, remove, validate, exit
myshell> validate
All job records are valid.
myshell> exit

Cleaning up jobs...
All resources cleaned up.
Exiting...
rishill10@ASUS16X:~/Week8_Skill$ |

```